



DOCUMENTO TÉCNICO

Casos de Uso — Técnico / QA

Actores, flujos, excepciones y componentes de cada proceso de la plataforma



ARIA · BY NOVASYS · 2026-07-10

Introducción

Este documento describe los **casos de uso** de la plataforma ARIA (Connectview) en formato estructurado para desarrollo y control de calidad. Cada caso indica actores, precondiciones, disparador, flujo principal, flujos alternativos/excepciones, postcondiciones y los componentes técnicos que intervienen (frontend, Lambdas, tablas, servicios AWS).

Convenciones. Los identificadores siguen el patrón `CU-T-NN`. Los "componentes" referencian el código real del repositorio. El modelo es **multi-tenant BYO** (Bring Your Own cloud): salvo la identidad (Cognito de ARIA), los recursos de datos, voz, WhatsApp y Bedrock viven en la cuenta AWS del cliente.

Ámbito	Casos
Identidad y onboarding	CU-T-01 · CU-T-02 · CU-T-03 · CU-T-04
Captación e ingesta	CU-T-05 · CU-T-06
Atención omnicanal	CU-T-07 · CU-T-08 · CU-T-09
Automatización e IA	CU-T-10 · CU-T-11 · CU-T-12
Salida y cumplimiento	CU-T-13 · CU-T-14
Analítica e integración	CU-T-15 · CU-T-16

CU-T-01 · Registro y aprovisionamiento de la organización

Actor principal: Usuario nuevo (futuro Admin). · **Sistema:** Cognito, `provision-tenant`. **Precondiciones:** ninguna (alta pública). **Disparador:** el usuario completa el formulario de registro (email, contraseña, nombre, empresa).

Flujo principal

1. El usuario se registra vía Amplify Authenticator.
2. Cognito crea el usuario y confirma el correo.
3. `provision-tenant` crea la organización (`tenantId`), asigna el grupo `Admins` y siembra la configuración inicial.
4. `fetchAuthSession()` devuelve el ID Token con `custom:tenantId` y grupos.
5. La app carga la UI según el rol.

Flujos alternativos / excepciones

- 2a. Email ya registrado → Cognito rechaza; se muestra "cuenta existente".
- 3a. Falla el aprovisionamiento → la sesión queda sin `tenantId` ; la app bloquea el acceso y ofrece reintentar.

Postcondiciones: existe una organización con un Admin; el usuario queda autenticado. **Componentes:** `VoxAuthContext` ; Lambda `provision-tenant` ; Cognito User Pool.

CU-T-02 · Inicio de sesión y apertura del softphone

Actor principal: Usuario registrado. · **Sistema:** Amazon Connect Streams, `get-federation-token`. **Precondiciones:** cuenta Cognito válida. **Disparador:** el usuario abre ARIA con sesión vigente, o pulsa "Conectar".

Flujo principal

1. `fetchAuthSession()` valida la sesión y entrega el ID Token.
2. La app carga la UI del rol (Agente / Supervisor / Admin).
3. Al pulsar "Conectar", `initCCP()` inicializa Amazon Connect Streams.
4. Si hay federación, `get-federation-token` hace SSO silencioso; si no, se abre el popup de login de Connect.
5. El softphone queda listo para voz/chat.

Flujos alternativos / excepciones

- 4a. Instancia sin SAML → no hay federación; se usa el popup (persistencia `loginPopup:true`).
- 5a. Falla la conexión de voz → `softphoneUnavailable = true` ; **la app sigue operando** sin voz (canales digitales intactos).
- 3a. Softphone ya abierto en otra pestaña → guard multi-pestaña (Web Locks + BroadcastChannel); la pestaña secundaria ofrece "Usar aquí".

Postcondiciones: sesión activa; softphone disponible o degradado de forma controlada. **Componentes:**

`ConnectAuthContext` , `CCPContext` , `src/lib/connect.ts` ; Lambda `get-federation-token` .

CU-T-03 · Onboarding BYO de un tenant

Actor principal: Admin del cliente. · **Sistema:** CloudFormation (cuenta cliente), `manage-connections` , `diagnose-connection` . **Precondiciones:** el cliente tiene cuenta AWS y una instancia de Amazon Connect. **Disparador:** el Admin inicia "Conectar Amazon Connect" en Integraciones.

Flujo principal

1. ARIA genera un `ExternalId` (CSPRNG) y una URL "Launch Stack".
2. El Admin aplica `connect-role.yaml` en **su** cuenta → se crea el rol `VoxCrmConnectAccess` .
3. El Admin pega el Role ARN y el ARN de la instancia de Connect.
4. ARIA guarda la config del tenant en `connectview-connections` .
5. El Admin pulsa "Verificar" → `diagnose-connection` asume el rol y comprueba tablas, S3 y Contact Lens.
6. ARIA muestra "✅ Conectado (N OK · 0 error)".

Flujos alternativos / excepciones

- 5a. Rol mal configurado / permisos faltantes → el diagnóstico lista cada checo con su error puntual.
- 4a-opt. BYO Data Plane → el Admin aplica `data-plane.yaml` (crea las tablas en su cuenta).

Postcondiciones: el tenant queda conectado; ARIA opera sobre los recursos del cliente sin custodiarlos.

Componentes: `IntegrationsManager` , `ConnectSetupWizard` , `cfnTemplates.ts` ; Lambdas `manage-connections` , `verify-connect-connection` , `diagnose-connection` .

CU-T-04 · Conectar Salesforce y WhatsApp

Actor principal: Admin. · **Sistema:** `salesforce-oauth-*` , `manage-connections` , Meta WABA. **Precondiciones:** tenant conectado (CU-T-03). **Disparador:** el Admin abre una integración adicional.

Flujo principal

1. **Salesforce:** OAuth (`salesforce-oauth-start` → `callback`); ARIA guarda tokens en Secrets Manager y descubre el esquema (`describeSObject`) para el mapeo de campos.

2. **WhatsApp:** el Admin registra el número y el WABA (modo AWS End User Messaging o Meta directo); ARIA valida y guarda la config del número.

Flujos alternativos / excepciones

- 1a. Token revocado → la sincronización falla suave y avisa "reconectar Salesforce".
- 2a. Varios números / cuentas Meta → cada número tiene su flujo (vista Ruteo).

Postcondiciones: integraciones activas; el mapeo ARIA→Salesforce por tenant queda listo. **Componentes:**

`SfFieldMapper` , `metaAccounts.ts` , `whatsappNumbers.ts` ; Lambdas `salesforce-oauth-start/callback` , `manage-connections` .

CU-T-05 · Ingesta automática de un lead (Meta Lead Ads)

Actor principal: Cliente potencial (rellena un formulario). · **Sistema:** `meta-lead-ads-webhook` , motor de automatizaciones. **Precondiciones:** página de Meta conectada; formulario de Lead Ads publicado. **Disparador:** Meta emite un evento `leadgen` .

Flujo principal

1. `meta-lead-ads-webhook` recibe el `leadgen` y normaliza los datos.
2. `propagateLead` crea/actualiza el lead y lo asocia a su Programa (auto-tag).
3. Se dispara la automatización de **speed-to-lead** (primer contacto inmediato).
4. El lead aparece en el embudo y en la bandeja del asesor.

Flujos alternativos / excepciones

- 1a. Teléfono duplicado → dedupe por teléfono / `VoxLeadId` ; se fusiona el historial.
- 3a. Fuera de horario → la automatización agenda el primer golpe para la ventana válida.

Postcondiciones: lead registrado, clasificado y en cola de atención; sin Zapier de por medio. **Componentes:**

`QuickCapture` ; Lambdas `meta-lead-ads-webhook` , `manage-leads` , `automation-engine` .

CU-T-06 · Atención de un contacto entrante (agente)

Actor principal: Agente. · **Actores secundarios:** Cliente final, Bedrock. · **Sistema:** Amazon Connect, Lambdas de contacto. **Precondiciones:** softphone conectado; contacto ruteado a la cola del agente. **Disparador:** entra un contacto (voz, chat, WhatsApp o email).

Flujo principal

1. Connect entrega el contacto por Streams → overlay entrante en el Workspace.
2. El agente acepta.
3. `get-contact-detail` / `get-customer-thread` cargan la vista 360° (perfil, historial, transcripción).
4. Durante el contacto, Contact Lens transcribe en vivo y el copiloto (`generate-call-summary`) sugiere réplicas y próxima acción.
5. El agente ejecuta acciones (transferir, conferencia, agendar callback, crear lead).
6. Al terminar, hace wrap-up: tipificación + notas (`save-agent-notes`).

Flujos alternativos / excepciones

- 4a. Sin transcripción disponible → el copiloto trabaja con lo que haya; degradación suave.
- 5a. Cliente en DNC → la acción de voz saliente queda bloqueada (ver CU-T-14).

Postcondiciones: contacto cerrado; perfil e historial actualizados; sync opcional a Salesforce. **Componentes:**

`AgentDesktopPage` ; Lambdas `get-contact-detail` , `get-live-transcript` , `generate-call-summary` , `save-agent-notes` , `update-customer-profile` .

CU-T-07 · Bot de WhatsApp entrante

Actor principal: Cliente (WhatsApp). · **Sistema:** End User Messaging (cuenta cliente), `agent-channel-adapter` , `bot-runtime` , Bedrock del cliente. **Precondiciones:** número de WhatsApp del tenant conectado; bot publicado.

Disparador: el cliente escribe por WhatsApp.

Flujo principal

1. El mensaje entra a End User Messaging y al Contact Flow de Connect.
2. El flujo invoca `agent-channel-adapter` con `{tenantId, contactId, mensaje}` .
3. El adapter llama a `bot-runtime` con `{botId, state, input, source: whatsapp}` .
4. `bot-runtime` recorre el grafo del bot (nodos/aristas).
5. En un nodo de IA (`ai_agent`), invoca el Bedrock del cliente con RAG (catálogos, programas, FAQ) y devuelve respuesta con citas.
6. El adapter envía la respuesta **desde el número del cliente**.

Flujos alternativos / excepciones

- 5a. Sin match en la base → respuesta de fallback controlada.
- 6a. El bot decide derivar → `handoff` a la cola de agentes (ver CU-T-08).

Postcondiciones: conversación avanzada o resuelta; estado persistido en `connectview-ai-conversations` .

Componentes: Lambdas `agent-channel-adapter` , `bot-runtime` ; tablas `connectview-bots` , `connectview-ai-conversations` ; `resolveBedrock` / `resolveWhatsApp` .

CU-T-08 · Escalar de bot a agente humano (handoff)

Actor principal: Bot / Agente IA. · **Actor secundario:** Agente humano. **Precondiciones:** conversación de bot en curso.

Disparador: el bot detecta intención de "hablar con asesor", baja confianza o una regla de negocio.

Flujo principal

1. `bot-runtime` marca `handoff` en el resultado.
2. El adapter encola el contacto hacia agentes con el contexto de la conversación.
3. El agente recibe el contacto ya con el historial del bot cargado (360°).
4. El agente continúa la atención (CU-T-06).

Flujos alternativos / excepciones

- 2a. Fuera de horario / sin agentes → se ofrece agendar callback o continuar por bot.

Postcondiciones: el humano toma el control sin pérdida de contexto. **Componentes:** `agent-channel-adapter` , `bot-runtime` , colas de Connect.

CU-T-09 · Campaña de salida (voz / WhatsApp)

Actor principal: Supervisor / Admin. · **Sistema:** `campaign-dialer`, EventBridge, Amazon Connect. **Precondiciones:** audiencia disponible; agentes asignados. **Disparador:** la campaña pasa a estado `RUNNING`.

Flujo principal

1. El supervisor carga audiencia (CSV / leads / pegar números), configura canal (voz predictivo/power/preview, WhatsApp, email) y reglas (horario, reintentos).
2. `create-campaign` persiste la campaña y sus contactos.
3. EventBridge (~1 min) dispara `campaign-dialer`, que hace ticks sub-minuto respetando pacing y % de abandono.
4. **Voz:** `StartOutboundVoiceContact` conecta al agente. **WhatsApp:** `send-whatsapp-template` envía desde el número del tenant.
5. El agente atiende; al cerrar, wrap-up (`save-agent-notes`).
6. `get-campaign-stats` muestra KPIs en vivo (contactados, conversión, AHT).

Flujos alternativos / excepciones

- 3a. Varias campañas activas → orquestación por prioridad + peso + metas (`computeSlotBudget`).
- 4a. Número en supresión → se salta (ver CU-T-14).

Postcondiciones: contactos gestionados; métricas disponibles; sync opcional a Salesforce. **Componentes:**

`CampaignCreatePage`, `CampaignBlendBoard`; Lambdas `create-campaign`, `control-campaign`, `campaign-dialer`, `get-campaign-stats`.

CU-T-10 · Resumen de llamada con IA

Actor principal: Agente. · **Sistema:** `generate-call-summary`, Bedrock del tenant. **Precondiciones:** contacto con transcripción (en vivo o histórica). **Disparador:** finaliza la llamada, o el agente abre un contacto histórico.

Flujo principal

1. Se obtiene la transcripción (Contact Lens en vivo, o inline para históricos).
2. `generate-call-summary` invoca `resolveBedrock` → Bedrock del tenant.
3. Claude genera resumen, tipificación sugerida, próxima acción y reescritura.
4. El agente ve la sugerencia en el panel de wrap-up / copiloto.

Flujos alternativos / excepciones

- 2a. Falla Bedrock → mensaje suave; el agente redacta manualmente (degradación).

Postcondiciones: wrap-up asistido; menor tiempo de post-llamada. **Componentes:** Lambda `generate-call-summary` (modos `summary`, `wrap-up-suggest`, `next-action`, `suggest-replies`, `rewrite`, `assistant`).

CU-T-11 · Automatización basada en reglas (trigger → acción)

Actor principal: Admin (configura) · Sistema (ejecuta). · **Sistema:** `automation-engine`, `connectview-automation-rules`. **Precondiciones:** al menos una regla activa. **Disparador:** ocurre un evento de dominio (nuevo lead, cambio de etapa, mensaje entrante, etc.).

Flujo principal

1. El evento entra al motor de automatizaciones.
2. El motor evalúa las condiciones de las reglas activas.
3. Ejecuta las acciones (notificar agente, enviar plantilla, mover etapa, crear tarea, iniciar journey), resolviendo tokens `[[name]]` / `{{name}}` .

Flujos alternativos / excepciones

- 2a. Ninguna regla coincide → no-op.
- 3a. Acción a canal en supresión → se omite.

Postcondiciones: efectos aplicados; traza en el ledger del lead. **Componentes:** Lambda `automation-engine` ; tabla `connectview-automation-rules` .

CU-T-12 · Journey multi-paso

Actor principal: Admin (diseña) · Sistema (ejecuta). · **Sistema:** `journey-runner` , EventBridge (5 min). **Precondiciones:** journey publicado; contacto inscrito. **Disparador:** una automatización o evento ejecuta `start_journey` .

Flujo principal

1. El contacto se inscribe en el journey (paso inicial).
2. Cada 5 min, `journey-runner` avanza a los contactos según esperas y condiciones.
3. En cada paso se ejecutan acciones (mensaje, espera, ramificación, salida).

Flujos alternativos / excepciones

- 2a. Condición de salida temprana (conversión, opt-out) → el contacto abandona el journey.

Postcondiciones: recorrido completado o abandonado; eventos registrados. **Componentes:** Lambda `journey-runner` ; tablas de journeys; `_shared/journeys.ts` .

CU-T-13 · Cierre de conversación

Actor principal: Agente / Agente IA / Sistema. · **Sistema:** `closeConversation` , reaper. **Precondiciones:** conversación abierta. **Disparador:** el agente cierra, el Agente IA marca `done` , o el reaper detecta inactividad (~10 min).

Flujo principal

1. Se invoca `closeConversation` de forma unificada.
2. Se marca la conversación como cerrada y se consolidan métricas.

Flujos alternativos / excepciones

- 3a. Reaper: cierre automático por inactividad para no dejar conversaciones colgadas.

Postcondiciones: conversación cerrada de forma consistente por cualquiera de los 3 disparadores. **Componentes:** `_shared/conversations.ts` .

CU-T-14 · Supresión / No molestar (cumplimiento)

Actor principal: Sistema. · **Sistema:** motor de supresión (`_shared/suppression.ts`), PK por teléfono. **Precondiciones:** lista de supresión poblada (DNC, opt-out de Salesforce, post-conversión). **Disparador:** cualquier

intento de contacto saliente (voz / WhatsApp).

Flujo principal

1. Antes de un golpe saliente, se consulta la supresión por teléfono.
2. Si el número está suprimido, se bloquea el contacto y se registra el motivo.

Flujos alternativos / excepciones

- 1a. Servicio de supresión no disponible → política **fail-open** (no bloquea, pero avisa) para no frenar la operación.
- 1b. No contactar tras conversión / opt-out en Salesforce → mismo bloqueo.

Postcondiciones: cumplimiento aplicado antes de molestar al cliente. **Componentes:** `_shared/suppression.ts` (bundleada en los 6 Lambdas de contacto saliente).

CU-T-15 · Reportes y exportación

Actor principal: Supervisor / Admin. · **Sistema:** Lambdas de reportes, `reportExport.ts`. **Precondiciones:** actividad registrada. **Disparador:** el usuario abre Reportes o pide una descarga.

Flujo principal

1. La vista de Reportes carga KPIs por dominio, con comparación contra el período previo.
2. El usuario filtra por rango de fechas / programa.
3. Descarga uno de los reportes (Excel/CSV) o consume el feed de Power BI.

Flujos alternativos / excepciones

- 3a. Feed Power BI → token HMAC + `?dataset=` ; sin almacenamiento de credenciales.

Postcondiciones: información disponible para decisión y para BI externo. **Componentes:** `reportExport.ts` , `get-bot-report` , `get-analytics-feed` .

CU-T-16 · Sincronización con Salesforce (write-back)

Actor principal: Sistema. · **Sistema:** `salesforce-sync` , mapeo schema-aware por tenant. **Precondiciones:** Salesforce conectado y mapeo definido (CU-T-04). **Disparador:** un golpe o cierre relevante (gestión, conversión, wrap-up).

Flujo principal

1. Tras el evento, `salesforce-sync` hace upsert de Lead/Task/campos `Vox*__c` .
2. Se aplica dedupe (teléfono + `VoxLeadId__c` + contactos convertidos).

Flujos alternativos / excepciones

- 1a. Campo destino inexistente en el org → el mapeo lo omite sin romper el sync.
- 1b. Token expirado → falla suave; se reintenta tras reconectar.

Postcondiciones: CRM del cliente actualizado con la actividad de ARIA. **Componentes:** Lambda `salesforce-sync` ; `SfFieldMapper` ; campos `Vox*__c` .

Trazabilidad y casos de prueba

Cada caso de uso de este documento se traza con **historias de usuario** (HU-NN), **criterios de aceptación** (en formato Dado/Cuando/Entonces) y **casos de prueba** (CP-NN con pasos y resultado esperado) en el libro de Excel adjunto:

ARIA-Historias-de-Usuario-y-Casos-de-Prueba.xlsx — hojas: *Historias de Usuario*, *Criterios de Aceptación*, *Casos de Prueba* y *Matriz de Trazabilidad* (CU ↔ HU ↔ CP).

La matriz permite verificar la cobertura: para cada **CU-T-NN** existe al menos una historia y un caso de prueba que la ejercita.

Documento generado sobre el código real del repositorio. Los componentes citados son verificables en **`amplify/functions/`** y **`src/`**.